Fakultas Informatika
School of Computing
Telkom University

Aplikasi Berbasis Platform

Web Framework (2)

Database configuration

- DB configuration file is stored in config/database.php. But for standard config and easier access, we can set it in .env file
- Fill on the line DB_CONNECTION to DB_PASSWORD

```
...
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=training
DB_USERNAME=root
DB_PASSWORD=
...
```

Creating database and migration file

Create a database named **ecommerce** in the DBMS. To create the table, it can be done automatically (generated) by Laravel from the command prompt / console / terminal. The first step is to create the migration file as a schema from the table:

```
php artisan make:migration create_products_table
```

The **xxx_create_products_table.php** migration file will be created in the database/migrations folder.

Set the schema (table columns)

Edit the migration file in the **Schema::create()** function to define the columns we want.

```
Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->integer('price');
    $table->timestamps();
});
```

- The id() function: we define the 'id' column as PK with int type and auto-increment.
- string('name') function: we define column 'name' with varchar type
- integer('price'): we define the column 'price' with int type
- timestamps() function: we define column 'created_at' which will be filled automatically if data is created via ORM and 'updated_at' which will be filled automatically if data is updated via ORM

Generate table based on migration file

To create a table to the database from this schema, then type the following command at the command prompt / console / terminal:

```
php artisan migrate
```

If your database configuration is right, table **products** will be created in **ecommerce** database.

Query Builder

- Laravel provides three ways to access or manipulate data to the database: direct (raw) query, query builder, and ORM (Model). The direct query and query builder use the Illuminate library, while the ORM uses Eloquent, which is a class extending from Illuminate
- Example using raw query:

```
DB::statement("insert into products (id, name, price)
 values (1, 'ASUS', 15000000)");
```

- Example using query builder:

```
$prods = DB::table('products')->get();
```

Basic Model

- Model file is stored inside app/Models folder
- You can create the Model manually or generate it with script:

```
php artisan make:model Product
```

- You can create the Controller (-c) or the migration (-m) at the same time with the Model, or both (-cm)

```
php artisan make:model Product –m
```

Laravel Model Conventions & Rules

- Model **X** will automatically represent a database table named **xs**. For example, **Product** model will represent **products** table. If you have different table name, you must add this attribute in Model: **protected $table = 'nama_tabel';**
- By default, PK column is represented with **id**. If you have different PK column name, you must add this attribute in Model: **protected $primaryKey = 'nama_kolom_PK';**
- By default, PK column is an integer. If you have different PK column type, you must add this attribute in Model: **protected $keyType = 'string';**
- By default, PK column is set to auto-increment. If it is not, you must add this attribute in Model: **public $incrementing = false;**
- By default, represented table has **created_at** and **updated_at** column. If it doesn't (in migration file you don't use **$table->timestamps()**), you must add this attribute in Model: **public $timestamps = false;**

Using Model in Controller

```
//get all products
$prods = Product::get();

//get one product by id
$prod = Product::find($id);

//insert/update new product
$prod = new Product; //use get one product for update
$prod->name = $request->name;
$prod->price = $request->price;
$prod->save();

//delete product
Product::destroy($id);
```

Model Relationship

- We can relate two Models to represent the cardinality of their tables
- With this, we can get the corresponding data that is referenced by the foreign key column, without using join query or do a separate query
- Let's say we have Variant Model that represents 'variants' table. One product can have many variants, but one variant belongs to only one product.

Model Relationship (contd.)

- So in this case, we create migration file and model file, or we can create both at the same time

```
php artisan make:model Variant -m
```

- Set the schema for Variants table:

```
...
Schema::create('variants', function (Blueprint $table) {
  $table->id();
  $table->string('name');
  $table->text('description');
  $table->string('processor');
  $table->string('memory');
  $table->string('storage');
  $table->foreignId('product_id')->constrained();
  $table->timestamps();
});
...
```

Model Relationship (contd.)

- For foreign key mapping, we use 'foreignId' and 'constrained' if the entity table that referenced follows the Eloquent Model convention: PK column is "id" and database table name is ended with "s". If your schema doesn't follow these rules, the foreign key definition is like this:

```
...
$table->integer('product_id');
$table->foreign('product_id')->references('id_product')->on('product');
...
```

- 'foreign' defines the foreign key column, 'references' defines the column that referenced, and 'on' defines the table that referenced.

Model Relationship (contd.)

- Then we add the relation function in Product Model:

```
public function variants() {
  return $this->hasMany(Variant::class);
}
```

- And in Variant Model:

```
public function product() {
  return $this->belongsTo(Product::class);
}
```

Using Model Relationship in Controller

- We can call the model (data) in Controller with 2 ways:
  - Lazy loading: call the entity model first, call the relation model later
  - Eager loading: call the entity model and relation model together

Call Model Relationship (Lazy Loading)

```
$prods = Product::get();

foreach ($prods as $d) {
  //get all variants from a product
  foreach ($d->variants()->get() as $var) {
    //get the product of a variant
    echo $var->product->name;
  }
}
```

Call Model Relationship (Eager Loading)

```
$prods = Product::with('variants')->get();

foreach ($prods as $d) {
  //get all variants from a product
  foreach ($d->variants as $var) {
    //get the product of a variant
    echo $var->product->name;
  }
}
```

Any question?

References

- https://laravel.com/docs/

Fakultas Informatika
School of Computing
Telkom University

THANK YOU